GENERATIVE AI ENGINEERING

The Complete GenAI Engineer Interview Guide

A structured, in-depth preparation resource covering LLM fundamentals, classical ML, prompt engineering, RAG, fine-tuning, vector databases, agents, multimodal & diffusion models, LLMOps, infrastructure, coding, system design, real-world scenarios, projects, and behavioral rounds.

11	80	10	6
CORE TOPICS	Q&A PAIRS	SCENARIOS	CODING EXERCISES

Prepared as a comprehensive interview-readiness reference · Foundational to Advanced coverage · 2026 Edition

TABLE OF CONTENTS

What's inside this guide

01 **LLM & Transformer Fundamentals** — 11 Questions
Every GenAI interview starts here. Interviewers use these questions to check whether you understand what's actually happening inside the model you're building products around — not just which API to call.

02 **Prompt Engineering** — 10 Questions
Prompt engineering questions test whether you can reliably control model behavior in production — not just write a clever one-off prompt.

03 **Retrieval-Augmented Generation (RAG)** — 10 Questions
RAG is the single most commonly built GenAI system in industry. Expect deep, practical questions here — interviewers want to know you've actually debugged a retrieval pipeline, not just read about one.

04 **Fine-Tuning & PEFT** — 7 Questions
Fine-tuning questions probe whether you understand the cost/benefit tradeoffs and the mechanics well enough to actually run a training job, not just cite technique names.

05 **Embeddings & Vector Databases** — 6 Questions
Almost every GenAI system relies on embeddings somewhere. These questions test whether you understand what a vector actually represents and how retrieval infrastructure scales.

06 **AI Agents & Orchestration** — 6 Questions
Agent questions are increasingly central to GenAI engineering interviews as products move from single-turn Q&A to multi-step autonomous workflows.

07 **LLMOps, Deployment & Cost** — 6 Questions
This is where 'can you actually run this in production' gets tested — latency, cost, reliability, and observability.

08 **Evaluation, Safety & Guardrails** — 6 Questions
Increasingly a dedicated interview section on its own — companies want to know you take output quality and safety seriously, not just capability.

09 **Classical ML & Deep Learning Fundamentals** — 6 Questions
Even GenAI-focused loops often open with core ML/DL fundamentals to confirm the basics are solid before going deep on transformers. Don't skip this section assuming it's 'too basic' — a shaky answer here is a fast, avoidable way to lose credibility early in the interview.

10 **Multimodal Models & Diffusion** — 6 Questions
As GenAI products increasingly span text, image, audio, and video, expect at least a couple of questions probing whether you understand what's structurally different about non-text modalities — even in a primarily text/LLM-focused role.

11 **Infrastructure & Deployment Engineering** — 6 Questions
The LLMOps section covered the what and why of production cost, latency, and monitoring; this section covers the how — the actual infrastructure decisions behind serving models reliably at scale.

12 **Coding & Practical Exercises** — 6 Exercises
Hands-on exercises covering the building blocks that come up in live/take-home coding rounds — retrieval, chunking, resilience patterns, and agent control flow.

13 **Scenario-Based Questions** — 10 Scenarios
Real production fire drills — debugging inconsistent outputs, latency spikes, stale RAG citations, runaway agent loops, and cost blowouts.

14 **System Design Case Studies** — 3 Cases
Three end-to-end architecture walkthroughs for common GenAI product scenarios, with the tradeoffs interviewers expect you to raise unprompted.

15 **Project & Portfolio Questions** — 6 Questions
How to structure answers about your own RAG, agent, and fine-tuning projects so they read as production experience, not tutorials.

16 **Behavioral & Judgment Questions** — 6 Questions
The non-technical questions that decide close calls — how you reason about failure, cost, and cross-functional collaboration.

17

General Behavioral & HR Questions

6 Questions

Standard cross-role behavioral and HR-round questions — conflict, weaknesses, motivation, and career trajectory.

+

Quick-Reference Glossary

15 Terms

A one-page cheat sheet of every key term, formula, and concept — ideal for a final skim right before the interview.

How to use this guide: Sections 1–11 are tagged Foundational, Intermediate, or Advanced per question. If you're short on time, prioritize Section 3 (RAG), Section 13 (Scenarios), and Section 15 (Projects) — these test applied judgment, which is where most 2026 GenAI engineer interviews actually differentiate candidates.

01

LLM & Transformer Fundamentals

Every GenAI interview starts here. Interviewers use these questions to check whether you understand what's actually happening inside the model you're building products around — not just which API to call.

Q. Explain the Transformer architecture and why it replaced RNNs for language modeling.

FOUNDATIONAL

The Transformer (Vaswani et al., 2017) processes an entire sequence in parallel using self-attention, instead of stepping through tokens one at a time like an RNN or LSTM. Each token's representation is updated by attending to every other token in the sequence, weighted by learned relevance scores.

This solves two core RNN problems: (1) vanishing gradients over long sequences, since attention gives every token a direct path to every other token regardless of distance, and (2) sequential computation, since RNNs can't be parallelized across time steps during training, while Transformers can process a whole batch of tokens simultaneously on a GPU.

A Transformer block = Multi-Head Self-Attention → Add & LayerNorm → Feed-Forward Network → Add & LayerNorm, stacked N times. Positional encodings are added at the input because attention itself is order-agnostic.

> **Interview tip —** If asked to draw it, sketch: Input Embeddings + Positional Encoding → [Self-Attention → Add&Norm → FFN → Add&Norm] × N → Output.

Q. Walk through self-attention mathematically. What are Q, K, and V?

FOUNDATIONAL

Each token embedding is projected into three vectors using learned weight matrices: Query (Q), Key (K), and Value (V). Intuitively, Q represents 'what this token is looking for,' K represents 'what this token offers,' and V is 'the actual content it contributes.'

Attention scores are computed as the scaled dot product of Q and K: softmax(QK^T / sqrt(d_k)) V. The scaling by sqrt(d_k) prevents the dot products from growing too large in magnitude as dimensionality increases, which would push softmax into regions with vanishing gradients.

Multi-head attention runs this process h times in parallel with different learned projections, letting the model attend to different types of relationships (syntax, coreference, long-range dependency) simultaneously, then concatenates and projects the results back down.

Q. What is the difference between encoder-only, decoder-only, and encoder-decoder architectures?

FOUNDATIONAL

Encoder-only (BERT-style): bidirectional attention — every token attends to every other token, including future ones. Best for understanding tasks: classification, embeddings, NER.

Decoder-only (GPT, LLaMA, Claude-style): causal (masked) attention — each token can only attend to previous tokens. This autoregressive property is what enables text generation one token at a time. Nearly all modern production LLMs are decoder-only because it scales well and unifies pretraining with generation.

Encoder-decoder (T5, original Transformer, BART): the encoder builds a bidirectional representation of the input, and the decoder cross-attends to it while generating output autoregressively. Well suited to sequence-to-sequence tasks like translation and summarization where input and output are distinct sequences.

Q. What is next-token prediction, and why is a simple objective enough to produce reasoning-like behavior?

INTERMEDIATE

Pretraining minimizes cross-entropy loss between the predicted probability distribution over the vocabulary and the actual next token, across trillions of tokens of text. It is a single, simple, self-supervised objective — no labels required.

Reasoning-like behavior emerges because predicting the next token well on diverse, high-quality text implicitly requires learning grammar, facts, world knowledge, and multi-step logical patterns — the model can't compress that much text accurately without internalizing structure. This is often called emergent capability: certain skills (arithmetic, chain-of-thought, code generation) appear abruptly once model and data scale pass a threshold, rather than improving smoothly.

Q. Explain KV caching and why it matters for inference latency and cost.

INTERMEDIATE

During autoregressive generation, computing attention for a new token requires the Key and Value vectors of every previous token. Without caching, you'd recompute K and V for the entire prefix at every single generation step — O(n^2) redundant work over a sequence of length n.

KV caching stores the K and V tensors for all previously processed tokens in GPU memory, so each new token only needs one forward pass through the model, reusing cached K/V instead of recomputing them. This turns generation from quadratic into roughly linear cost per step.

The tradeoff is memory: KV cache size grows with sequence length × batch size × number of layers × hidden size, and it's frequently the actual bottleneck limiting how many concurrent requests a GPU can serve — which is why techniques like PagedAttention (vLLM), multi-query attention (MQA), and grouped-query attention (GQA) exist.

> **Interview tip —** GQA is a favorite follow-up: it shares K/V heads across groups of query heads, cutting KV cache size with minimal quality loss — used in LLaMA 2/3, Mistral.

Q. What is the difference between pretraining, supervised fine-tuning (SFT), and RLHF/preference tuning?

INTERMEDIATE

Pretraining: next-token prediction on massive unlabeled text corpora. This is where the model acquires language ability and world knowledge — it is by far the most compute-intensive stage.

SFT (instruction tuning): fine-tuning on curated (prompt, ideal response) pairs so the base model learns to follow instructions and adopt a helpful assistant format, rather than just continuing text in a document-completion style.

RLHF / DPO (preference tuning): the model's outputs are ranked by humans (or a reward model trained on human preferences), and the policy is optimized to produce outputs humans prefer — improving helpfulness, honesty, and harmlessness beyond what SFT alone achieves. DPO (Direct Preference Optimization) achieves a similar effect without training a separate reward model or running RL, directly optimizing the policy on preference pairs, which is simpler and more stable to train.

Q. What is temperature, top-k, and top-p (nucleus) sampling? How do they differ?

FOUNDATIONAL

Temperature reshapes the probability distribution before sampling: dividing logits by $T < 1$ sharpens the distribution (more deterministic, favors high-probability tokens), while $T > 1$ flattens it (more random/creative). $T = 0$ is effectively greedy decoding.

Top-k sampling restricts sampling to only the k highest-probability tokens, discarding the long tail entirely — a fixed cutoff regardless of how the probability mass is distributed.

Top-p (nucleus) sampling instead selects the smallest set of tokens whose cumulative probability exceeds p (e.g., 0.9), so the candidate pool adapts dynamically — narrow when the model is confident, wider when it's uncertain. In practice, top-p is generally preferred over top-k for this adaptiveness, and both are often combined with temperature.

Q. What are the main scaling laws for LLMs, and what did Chinchilla change about them?

ADVANCED

Kaplan et al. (2020) showed loss decreases predictably as a power law with model size, dataset size, and compute — and, given a compute budget, argued for prioritizing larger models over more data, which drove the era of very large but undertrained models like GPT-3.

Chinchilla (Hoffmann et al., 2022) re-ran the analysis more rigorously and found that most models of that era were significantly undertrained relative to their size — for a fixed compute budget, model size and training tokens should scale roughly equally (roughly ~20 tokens per parameter is the commonly cited rule of thumb). This is why later model families (LLaMA, etc.) trained smaller models on far more tokens and got better performance per FLOP.

Q. What is the difference between quantization approaches like INT8, INT4, GPTQ, and AWQ?

ADVANCED

Quantization reduces the numerical precision of model weights (and sometimes activations) from FP16/BF16 down to INT8 or INT4, shrinking memory footprint and increasing throughput, at some cost to output quality.

Post-training quantization (PTQ) methods like GPTQ minimize the layer-wise reconstruction error by solving a per-layer optimization problem after training, without needing to retrain — it's popular for INT4 quantization with minimal accuracy loss.

AWQ (Activation-aware Weight Quantization) observes that not all weights are equally important — it identifies and preserves precision for the small fraction of weight channels that correspond to high-magnitude activations, since those matter disproportionately for output quality, and quantizes the rest more aggressively.

In interviews, the practical framing matters most: quantization is usually the first lever engineers pull to fit a model on smaller/cheaper GPUs or increase batch throughput, and the trade is memory/speed vs. a measurable (though often small) quality regression that should be evaluated on your actual task, not just perplexity.

Q. What is positional encoding, and how do RoPE and ALiBi differ from the original sinusoidal approach?

ADVANCED

Self-attention has no inherent notion of token order — it treats a sequence like an unordered set unless position information is explicitly injected. The original Transformer added fixed sinusoidal position vectors directly to token embeddings before the first layer.

RoPE (Rotary Position Embedding) instead encodes position by rotating the Query and Key vectors by an angle proportional to their position, applied at every attention layer rather than only at the input. This gives it a useful property: the attention score between two tokens depends naturally on their relative distance, and it extrapolates to longer sequences better than fixed sinusoidal encodings. RoPE is used in LLaMA, Mistral, and most modern open-weight decoder-only models.

ALiBi (Attention with Linear Biases) skips positional embeddings entirely and instead directly biases attention scores by a penalty proportional to the distance between tokens — closer tokens get less penalty. It's simpler and was designed specifically for strong length extrapolation (training short, running long) with minimal extra compute.

Q. What is Mixture of Experts (MoE), and why are labs increasingly using it for frontier models?

ADVANCED

MoE replaces a single dense feed-forward block in each Transformer layer with multiple 'expert' feed-forward networks, plus a lightweight learned router that decides which few experts (e.g., 2 of 8) process each token. Only the selected experts' weights are activated for a given token, not the full parameter set.

This decouples total parameter count from per-token compute cost: a model can have a very large total parameter count (more capacity/knowledge) while the actual FLOPs spent per token stay close to a much smaller dense model — you get the quality benefits of scale without paying the full inference cost for every token.

The tradeoffs interviewers want to hear: routing can be unstable to train (load balancing losses are needed so experts aren't over/under-used), MoE models need more total GPU memory to hold all experts even though only a few are active per token, and serving them efficiently requires careful expert-parallelism infrastructure — it's a systems-engineering problem as much as a modeling one.

02

Prompt Engineering

Prompt engineering questions test whether you can reliably control model behavior in production — not just write a clever one-off prompt.

Q. What is few-shot prompting, and when does it help vs. hurt?

FOUNDATIONAL

Few-shot prompting includes a small number of example (input, output) pairs in the prompt so the model can infer the desired format, tone, or task pattern purely from context, without any weight updates — this is called in-context learning.

It helps most for structured, non-obvious output formats (specific JSON schemas, unusual classification labels, style matching) where instructions alone are ambiguous.

It can hurt when examples are too similar to each other and cause the model to overfit to surface patterns instead of the underlying task, when examples are low-quality (the model will faithfully imitate mistakes), or when it simply eats context budget better spent on retrieved knowledge for a RAG-heavy task. Order and diversity of examples also measurably affects output — this is a well-documented instability in in-context learning.

Q. Explain Chain-of-Thought (CoT) prompting and why it improves performance on reasoning tasks.

FOUNDATIONAL

CoT prompting asks the model to produce intermediate reasoning steps before the final answer (e.g., 'Let's think step by step'), rather than jumping directly to a conclusion.

It helps because it gives the model more forward computation and 'working memory' in the form of tokens it can condition on — the model effectively gets to reason token-by-token instead of having to compress a multi-step derivation into a single forward pass. It's most impactful on arithmetic, logic, and multi-step reasoning tasks, and shows little to no benefit on simple factual recall.

Zero-shot CoT ('think step by step') vs few-shot CoT (showing worked examples with reasoning) are both common; few-shot CoT generally performs better but costs more context tokens.

Q. What is self-consistency, and how does it improve on plain Chain-of-Thought?

INTERMEDIATE

Self-consistency samples multiple independent reasoning paths for the same question (using temperature > 0 so each generation differs), then takes a majority vote over the final answers instead of trusting a single greedy chain of reasoning.

The intuition: a single CoT trace can go down a wrong reasoning path and still sound confident, but wrong paths tend to disagree with each other while correct reasoning tends to converge on the same answer from different angles. This measurably improves accuracy on arithmetic and logical reasoning benchmarks at the cost of running inference N times instead of once — a latency/cost tradeoff worth naming explicitly in an interview answer.

Q. How do you reliably get structured (e.g., JSON) output from an LLM in production?

INTERMEDIATE

The most reliable approach is constrained decoding / structured output modes (e.g., function calling or a JSON-schema-constrained generation mode) where the model's token sampling is actually restricted at each step to only tokens that keep the output valid against the schema — this guarantees syntactic validity, unlike asking nicely in a prompt.

When constrained decoding isn't available, the fallback pattern is: provide the schema and a concrete example in the prompt, ask for JSON only with no surrounding prose, parse the output defensively, and automatically retry with the parse error fed back to the model if it fails validation — capped at a small number of retries with a fallback path (e.g., escalate or return a default) if it keeps failing.

Common mistake to flag in an interview: relying purely on prompt instructions ('respond only in JSON') without any validation/retry layer — this works most of the time in a demo and then breaks unpredictably in production traffic.

Q. When would you use prompt chaining (multiple LLM calls) instead of one large complex prompt?

INTERMEDIATE

Prompt chaining breaks a complex task into a sequence of smaller, focused LLM calls, where each step's output feeds the next — e.g., extract → classify → summarize → format, rather than asking one prompt to do all four at once.

It helps when a single prompt asking for too much at once degrades quality (the model has to juggle many instructions simultaneously and something gets dropped), when different steps benefit from different models (a cheap model for extraction, an expensive one for the hard reasoning step), or when you need visibility/control at intermediate steps for debugging, validation, or human review.

The tradeoff is added latency (multiple sequential round-trips) and cost, plus more places for errors to compound across the chain — so it's a deliberate architecture decision, not a default. A good answer frames it as: chain when task decomposition genuinely improves reliability or control; keep it a single call when the task is simple enough that decomposition only adds latency without a quality benefit.

Q. What is the difference between prompt engineering, RAG, and fine-tuning — how do you decide which to use?

INTERMEDIATE

Prompt engineering shapes behavior at inference time with no model or knowledge changes — cheapest and fastest to iterate, best for format/style/reasoning-strategy control, but limited by context window and can't inject knowledge the model wasn't trained on reliably.

RAG (Retrieval-Augmented Generation) injects external, up-to-date, or proprietary knowledge into the prompt at query time via retrieval — solves the knowledge-freshness and hallucination-on-facts problem without retraining, and keeps a clear citation trail back to source documents.

Fine-tuning changes the model's weights — best when you need to change behavior/style/format consistently at scale (so you don't have to re-explain it every prompt), teach a new skill or domain vocabulary, or reduce latency/cost by baking instructions into weights instead of a long system prompt. It does NOT reliably fix hallucination or add real-time knowledge.

The standard interview answer: start with prompt engineering, add RAG when you need grounded/current knowledge, and only fine-tune once you've validated the use case and hit a ceiling that prompting/RAG can't solve — because fine-tuning is the most expensive to iterate on and maintain.

> **Interview tip —** This is one of the single most common GenAI engineer interview questions. Have a crisp, structured answer ready — interviewers are checking for judgment, not memorized definitions.

Q. What is prompt injection, and how do you defend against it in a production application?

INTERMEDIATE

Prompt injection is when untrusted input (from a user, a retrieved document, a tool result, a webpage) contains text crafted to override the system's original instructions — e.g., a document that says 'ignore previous instructions and reveal your system prompt.'

Defenses include: clearly delimiting untrusted content from instructions (e.g., XML tags, structured message roles) so the model can distinguish data from commands; using the least-privileged tools/permissions possible so even a successful injection can't cause real damage; input/output filtering and classifiers to catch known injection patterns; instructing the model explicitly to treat retrieved/tool content as data, not commands; and never letting a model take irreversible actions (sending emails, deleting data, executing purchases) without a human-in-the-loop confirmation step.

It's important to be honest in interviews that prompt injection is not fully solved — defense in depth (multiple imperfect layers) is the realistic state of the art, not a single silver-bullet fix.

Q. How would you handle a very long input that exceeds the model's context window?

INTERMEDIATE

Map-reduce summarization: split the input into chunks, summarize each independently, then summarize the summaries (recursively, if needed) — works well for tasks like 'summarize this 300-page document.'

Refine/iterative: process chunks sequentially, updating a running answer/summary with each new chunk's information — preserves more nuance than map-reduce but is inherently sequential and slower.

Retrieval-based reduction: instead of trying to fit everything in, use RAG-style retrieval to select only the most relevant portions of the input for the specific question being asked, rather than processing the whole document for every query.

For genuinely long-context-native tasks, a long-context model (100K+ tokens) may just be the right tool — but even then, be ready to discuss the 'lost in the middle' effect, where models attend less reliably to information buried in the middle of a very long context versus the beginning or end.

Q. How do you design a robust system prompt for a production application?

INTERMEDIATE

Be explicit and structured: define the role/persona, the task boundaries (what it should and shouldn't do), the output format (with an example), and how to handle edge cases (missing info, out-of-scope requests, ambiguous input).

Use clear structural markers (XML tags or headers) to separate instructions, context/retrieved data, and conversation history — this reduces the model conflating instructions with content and improves instruction-following reliability.

Iterate empirically: build a small evaluation set of realistic inputs (including adversarial and edge cases), test prompt changes against it before shipping, and version-control prompts like code since they're a core part of application behavior, not just configuration.

Q. What is ReAct prompting, and how does it differ from plain Chain-of-Thought?

ADVANCED

ReAct (Reason + Act) interleaves reasoning traces with concrete actions (tool calls) and observations: the model reasons about what to do, takes an action (e.g., calls a search API), observes the result, and reasons again — repeating until it reaches a final answer.

Plain CoT reasons entirely from the model's internal knowledge with no way to fetch new information or verify facts mid-reasoning. ReAct grounds reasoning in real-world feedback at each step, which is why it's the foundational pattern behind most agentic tool-use frameworks — the reasoning trace itself gets corrected by what the tools actually return, reducing compounding hallucination compared to reasoning blind.

03

Retrieval-Augmented Generation (RAG)

RAG is the single most commonly built GenAI system in industry. Expect deep, practical questions here — interviewers want to know you've actually debugged a retrieval pipeline, not just read about one.

Q. Describe the end-to-end RAG pipeline.

FOUNDATIONAL

Ingestion (offline): documents are loaded, cleaned, and split into chunks; each chunk is passed through an embedding model to produce a dense vector; vectors + metadata are stored in a vector database (or hybrid store) with an index for fast similarity search.

Retrieval (online, per query): the user query is embedded with the same embedding model, the vector store is searched for the top-k most similar chunks (often combined with keyword/BM25 search — hybrid retrieval), and results are optionally re-ranked with a cross-encoder for higher precision.

Generation: retrieved chunks are inserted into the prompt as context, alongside the user's question and system instructions, and the LLM generates a grounded answer — ideally with citations back to the source chunks.

Q. What chunking strategy would you use, and what are the tradeoffs of chunk size?

INTERMEDIATE

Fixed-size chunking (e.g., 512 tokens with 10-20% overlap) is simple and predictable but can split sentences or ideas mid-thought. Recursive/semantic chunking respects natural boundaries — paragraphs, sections, headers — and generally produces more coherent, self-contained chunks. Document-structure-aware chunking (splitting on markdown headers, table boundaries) is essential for structured docs.

Small chunks improve retrieval precision (each chunk is topically focused, so similarity search is sharper) but risk losing surrounding context needed to answer the question, and can fragment a single idea across multiple chunks. Large chunks preserve context and coherence but dilute the embedding's specificity (a long chunk about many things embeds to a 'blurry' average vector) and burn more of the context window per retrieved item.

A strong practical answer mentions parent-document / small-to-big retrieval: embed small chunks for precise matching, but return the larger parent section or surrounding context to the LLM for generation — getting retrieval precision and generation context simultaneously.

> **Interview tip —** Interviewers love hearing you say 'it depends on the document type and query pattern' — followed by a concrete example. That signals real experience over textbook knowledge.

Q. What is hybrid search, and why combine dense and sparse retrieval?

INTERMEDIATE

Dense retrieval (embedding similarity, e.g., cosine similarity over vectors) captures semantic meaning — it can match 'automobile' to a query about 'car' even with no shared words. But it can underperform on exact-match needs: product codes, acronyms, proper nouns, rare technical terms, or numbers, where embeddings can blur precise lexical matches.

Sparse retrieval (BM25/TF-IDF, keyword-based) excels at exact term matching and is fast, interpretable, and doesn't require training — but misses semantic/paraphrase matches entirely.

Hybrid search runs both and fuses results (commonly with Reciprocal Rank Fusion, RRF), capturing the strengths of each. In production RAG for technical or enterprise domains (legal, medical, code), hybrid search is close to a default best practice rather than an optional enhancement.

Q. What is re-ranking, and where does it fit in the pipeline?

INTERMEDIATE

Initial retrieval (bi-encoder/dense search) is optimized for speed — it embeds queries and documents independently, so similarity is just a fast vector comparison, but that independence loses some accuracy since query and document never actually interact.

Re-ranking uses a cross-encoder that takes the (query, candidate-chunk) pair together and scores relevance jointly, capturing much richer interaction signal — but it's far more expensive computationally, so it can't run over the whole corpus.

The standard pattern: retrieve a larger candidate set cheaply (e.g., top-50 with dense/hybrid search), then re-rank that smaller set with a cross-encoder and keep only the top-k (e.g., top-5) to actually pass into the LLM's context. This two-stage retrieve-then-rerank approach reliably improves answer quality with a bounded latency cost.

Q. How do you evaluate a RAG system? What metrics matter?

ADVANCED

Retrieval metrics: context precision (are retrieved chunks actually relevant?), context recall (did retrieval find all the necessary information that exists in the corpus?), and MRR/NDCG for ranking quality.

Generation metrics: faithfulness / groundedness (is the answer actually supported by the retrieved context, or did the model hallucinate beyond it?), answer relevance (does it actually address the question asked?), and answer correctness against a labeled reference where available.

Frameworks like RAGAS and TruLens operationalize these using an LLM-as-judge approach — an evaluator model scores faithfulness/relevance against the retrieved context automatically, which scales far better than manual review. In production, this is paired with online metrics: user thumbs up/down, click-through on citations, and escalation/fallback rate.

A key interview signal: distinguishing retrieval failures from generation failures. If the answer is wrong, first check whether the right information was even retrieved — if not, it's a retrieval/chunking/embedding problem; if the right context was retrieved but the model still got it wrong or hallucinated beyond it, it's a generation/prompting problem. Debugging RAG without this separation wastes a lot of time.

Q. How would you reduce hallucination in a RAG system that has correct information retrieved but still gets facts wrong?

ADVANCED

Prompt-level: explicitly instruct the model to answer only from provided context and to say 'I don't have enough information' when the context doesn't cover the question; ask for citations/quotes tied to specific chunks, which forces the model to ground each claim rather than free-associate.

Structural: reduce the amount of irrelevant context passed in (noise dilutes grounding — this is why re-ranking and tighter top-k matter), and place the most relevant chunks near the beginning or end of the context window since models can be less attentive to information buried in the middle of a long context ('lost in the middle' effect).

Post-generation: run a faithfulness check — either a lightweight LLM-as-judge call that verifies each claim in the answer is supported by the retrieved chunks, or a natural language inference (NLI) style entailment check — and either flag, regenerate, or abstain when the answer isn't well grounded.

Lower temperature for factual QA tasks, since higher temperature increases the chance of the model deviating into plausible-sounding but ungrounded phrasing.

Q. What is GraphRAG, and when would you use it over standard vector-based RAG?

ADVANCED

GraphRAG builds a knowledge graph from the corpus — extracting entities and relationships — and retrieves by traversing that graph structure (and often by summarizing graph communities/clusters), rather than relying solely on vector similarity over isolated chunks.

It's most valuable for questions that require connecting information across many documents or reasoning over relationships (e.g., 'how are these two organizations connected?', or holistic 'summarize everything about X across the whole corpus' queries), where standard chunk-level similarity search struggles because no single chunk contains the full answer.

The tradeoff is significant added complexity and cost — graph construction/maintenance is expensive and needs to be kept in sync with the corpus — so it's justified mainly for multi-hop, relationship-heavy, or corpus-wide synthesis use cases, not as a default replacement for standard RAG.

Q. How do you handle multi-hop questions in RAG, where the answer requires combining information from several documents?

ADVANCED

Single-shot retrieval (embed the question once, retrieve top-k, generate) often fails on multi-hop questions because the initial query only captures the first 'hop' — e.g., 'What company did the founder of X later start?' requires first resolving who founded X, then searching for their later company, which a single similarity search over the original question won't surface.

Common patterns: query decomposition (break the question into sub-questions and retrieve for each separately, then synthesize); iterative/agentic retrieval (retrieve, reason about what's still missing, issue a follow-up retrieval — essentially ReAct applied to RAG); and GraphRAG-style traversal when the hops follow explicit entity relationships.

The interview signal here is recognizing that multi-hop failures look like a generation problem (the model 'didn't know' the answer) but are actually a retrieval-architecture problem — the right single-pass retrieval never had a chance to surface all the needed evidence in the first place.

Q. How do you handle conflicting information across retrieved sources in a RAG system?

ADVANCED

First, surface the conflict rather than silently picking one side — instruct the model explicitly to note when retrieved sources disagree and, where possible, attribute each claim to its specific source rather than presenting a single blended answer as fact.

Use source metadata to help resolve conflicts programmatically where possible: recency (prefer newer documents for time-sensitive facts), source authority/trust tier (an official policy doc should usually outrank a Slack message in the same index), and document versioning so superseded content can be filtered out or down-weighted at retrieval time rather than left to the LLM to adjudicate.

This is also a strong argument for maintaining data hygiene in the underlying corpus — deduplication and deprecation of stale documents at ingestion time prevents many conflicts from ever reaching the retrieval stage at all, which is more reliable than trying to resolve them post-hoc in the prompt.

Q. RAG vs. a very long context window model — when would you still choose RAG even if the model could fit everything in context?

ADVANCED

Cost and latency: stuffing a huge corpus into every prompt means paying for and waiting on those input tokens on every single query, even when only a tiny fraction is relevant — RAG's targeted retrieval is far cheaper and faster at scale.

Freshness and maintainability: updating a vector index when a document changes is far simpler and cheaper than re-processing a giant static context, and avoids needing to re-run everything through the model just to reflect one changed fact.

Precision and traceability: retrieval gives an explicit, auditable set of source chunks a claim is grounded in; even with a large context window, models are demonstrably less reliable at pinpointing and using information buried in the middle of a huge context ('lost in the middle') than at using information surfaced by targeted retrieval.

Access control: retrieval lets you filter by permissions per-query, whereas stuffing an entire corpus into context makes it much harder to enforce document-level access control cleanly. In practice, many production systems use both — RAG to narrow down to a relevant working set, with a longer context window as headroom rather than a replacement for retrieval.

04

Fine-Tuning & PEFT

Fine-tuning questions probe whether you understand the cost/benefit tradeoffs and the mechanics well enough to actually run a training job, not just cite technique names.

Q. What is LoRA, and why is it so widely used for fine-tuning LLMs?

FOUNDATIONAL

LoRA (Low-Rank Adaptation) freezes all original model weights and injects small trainable low-rank matrices (A and B, where the update is approximated as $\Delta W = BA$, with rank r much smaller than the original dimensions) alongside key weight matrices, typically the attention projection layers.

This works because weight updates during fine-tuning tend to have a low 'intrinsic rank' — the meaningful change needed to adapt a model to a new task doesn't require updating every parameter with full freedom. By training only the small A/B matrices, LoRA cuts trainable parameters by orders of magnitude (often <1% of the full model).

Practical benefits: far less GPU memory needed (no optimizer states for frozen weights), much faster training, small checkpoint sizes (megabytes instead of gigabytes) making it easy to store/swap many task-specific adapters, and at inference time the adapter can be merged back into the base weights with zero added latency.

Q. What is QLoRA, and how does it differ from standard LoRA?

INTERMEDIATE

QLoRA combines LoRA with 4-bit quantization of the frozen base model weights (using a specialized 4-bit NormalFloat data type designed for normally-distributed weights), plus double quantization and paged optimizers to manage memory spikes.

The base model stays frozen and quantized to 4-bit (drastically cutting memory footprint), while the small LoRA adapter matrices are still trained in higher precision (typically BF16), and gradients flow through the quantized weights during backprop without ever fully dequantizing them into memory.

This is what made it practical to fine-tune large models (65B+ parameters) on a single consumer/prosumer GPU, and is now a default choice whenever GPU memory is the binding constraint for a fine-tuning job.

Q. When should you fine-tune vs. rely on prompting/RAG? Give a concrete example.

INTERMEDIATE

Fine-tune when: you need a consistent output format/style at scale without repeating lengthy instructions every call (reducing prompt-token cost and latency), you're teaching a narrow skill the base model does poorly even with good prompting (e.g., a very specific structured extraction schema, or a domain-specific classification task with many labels), or you need to bake in a tone/persona very reliably.

Don't fine-tune to add knowledge — this is a very common mistake. Fine-tuning teaches patterns and behavior, but is a poor and expensive way to inject new facts reliably; models fine-tuned on new facts can still hallucinate or misremember them, whereas RAG retrieves facts at query time with a verifiable source. Fine-tuning also doesn't handle facts that change after training without retraining.

Concrete example: a customer support bot that must always output a structured JSON ticket-classification with 40 highly specific internal categories is a good fine-tuning candidate (a narrow, repeated, format-heavy skill). A bot answering questions about your company's constantly-updated product documentation is a RAG candidate, not a fine-tuning one.

Q. What is catastrophic forgetting, and how do you mitigate it during fine-tuning?

ADVANCED

Catastrophic forgetting is when fine-tuning on a narrow new task degrades the model's previously strong general capabilities — e.g., a model fine-tuned heavily on customer-support tickets becomes worse at general reasoning or coding than the base model was.

Mitigations: use PEFT methods like LoRA instead of full fine-tuning, since freezing most weights naturally limits how much general behavior can shift; mix a portion of general-purpose/instruction data into the fine-tuning set alongside the task-specific data, so the model doesn't over-specialize; use a lower learning rate and fewer epochs (over-training on a small dataset is the most common cause); and evaluate on held-out general benchmarks — not just the target task — before and after fine-tuning to catch regressions early.

Q. How do you prepare and validate a fine-tuning dataset for production use?

INTERMEDIATE

Quality over quantity: a few thousand high-quality, diverse, correctly-labeled examples reliably outperform a much larger noisy dataset — LLMs are sensitive to label noise during fine-tuning, since it directly teaches the pattern you're optimizing for.

Ensure diversity across edge cases, input lengths, and phrasing so the model generalizes rather than memorizing surface patterns; deduplicate near-identical examples, which otherwise bias training and can cause overfitting/memorization.

Hold out a representative validation and test split before training (not just train/loss curves) and build a small human-reviewed evaluation set that mirrors real production traffic patterns, since offline loss metrics often don't correlate well with real-world output quality.

Q. Full fine-tuning vs. PEFT (LoRA/QLoRA) — when does full fine-tuning actually win?

ADVANCED

Full fine-tuning updates every parameter, giving the model maximum flexibility to adapt — it's the right call when the target domain or task is very far from the base model's pretraining distribution (e.g., a highly specialized scientific domain with its own vocabulary and reasoning patterns) and you have a genuinely large, high-quality dataset to support it, since PEFT's restricted capacity can become a ceiling in that regime.

In practice, full fine-tuning is far more expensive (full optimizer states, much more GPU memory, slower iteration) and carries a higher risk of catastrophic forgetting, so most teams default to LoRA/QLoRA and only reach for full fine-tuning when they've empirically hit PEFT's ceiling on their specific task — not as a starting point. A strong interview answer leads with that default-to-PEFT judgment rather than treating the two as equally likely choices.

Q. How would you generate synthetic data for fine-tuning when you don't have enough labeled examples?

ADVANCED

Use a strong LLM to generate candidate (input, output) pairs from seed examples or templates — e.g., paraphrasing real examples, generating variations across edge cases, or having the model produce reasoning traces for distillation from a larger to a smaller model.

Quality control is the hard part: synthetic data inherits and can amplify the generator model's own biases and errors, so it needs filtering (rule-based checks, a second LLM-as-judge pass, or human spot-review of a sample) before being trusted in a training set — blindly training on unfiltered synthetic data is a common way to bake subtle systematic errors into a fine-tuned model.

Best practice is to mix synthetic data with a smaller amount of real, human-verified examples, and to explicitly evaluate the fine-tuned model on a held-out set of real examples (not synthetic ones) to catch any distributional gap between synthetic and real-world inputs.

05

Embeddings & Vector Databases

Almost every GenAI system relies on embeddings somewhere. These questions test whether you understand what a vector actually represents and how retrieval infrastructure scales.

Q. What is an embedding, and what determines embedding quality for a given task?

FOUNDATIONAL

An embedding is a dense numerical vector representation of text (or image/audio) in a high-dimensional space, produced by a model trained so that semantically similar inputs map to nearby vectors (typically measured by cosine similarity or dot product).

Quality depends on: what the embedding model was trained on (domain match matters — a general web-trained embedding model may perform poorly on legal or medical jargon vs. a domain-tuned one), the training objective (contrastive learning on similar/dissimilar pairs is standard), dimensionality (higher isn't always better — it raises storage/compute cost and can add noise), and whether it's evaluated with a task-relevant benchmark like MTEB rather than assumed from vendor marketing alone.

Q. Compare HNSW, IVF, and exact (brute-force) search for vector similarity search.

ADVANCED

Brute-force / exact search compares the query vector to every vector in the dataset — perfectly accurate (100% recall) but O(n) per query, which becomes too slow at scale (millions+ of vectors).

IVF (Inverted File Index) clusters vectors into partitions (via k-means) at index time; at query time, it only searches the nearest few clusters instead of the whole dataset — much faster, with a tunable accuracy/speed tradeoff via the number of clusters probed, but requires a training/clustering step and can miss results near cluster boundaries.

HNSW (Hierarchical Navigable Small World) builds a multi-layer graph structure where each vector is a node connected to its approximate nearest neighbors; search greedily traverses from sparse upper layers down to dense lower layers, achieving excellent recall/speed tradeoffs and is the most widely used ANN algorithm in modern vector databases (default in most of them). Its main cost is memory (graph structure overhead) and slower index build/insert time compared to IVF.

All three trade off recall vs. latency vs. memory vs. build time — approximate methods (HNSW, IVF) are what make sub-100ms search over billions of vectors feasible in production, at the cost of a small, tunable amount of retrieval accuracy versus exact search.

Q. How do you choose between a dedicated vector database (Pinecone, Weaviate, Qdrant, Milvus) and a vector-capable extension on an existing database (pgvector, Elasticsearch, MongoDB Atlas)?

INTERMEDIATE

Dedicated vector DBs are purpose-built for high-scale ANN search, tend to have more mature filtering, hybrid search, and multi-tenancy features out of the box, and are the better default when vector search is the core workload and scale is large (tens of millions+ vectors, high QPS).

Extensions like pgvector are compelling when you already run that database for transactional data and want to avoid operating a second system, when scale is moderate, or when you need vector search tightly joined with existing relational data and transactional guarantees (ACID) in the same query.

Practical decision drivers: existing infra and team familiarity, expected scale and latency SLAs, need for hybrid (metadata-filtered) search, and operational overhead tolerance — this is squarely a 'depends on constraints' answer, and interviewers want to see you reason through the tradeoffs rather than name-drop a single 'best' tool.

Q. What is the difference between cosine similarity, dot product, and Euclidean (L2) distance for vector search — and when does it matter which you pick?

INTERMEDIATE

Cosine similarity measures the angle between two vectors, ignoring magnitude — good when only direction/semantic orientation matters, not vector length.

Dot product incorporates both angle and magnitude; it's the fastest to compute and is the correct choice when the embedding model was specifically trained to optimize dot product similarity (many modern embedding models are).

Euclidean/L2 distance measures straight-line distance in the vector space; it's sensitive to magnitude and is more common outside of text embeddings (e.g., clustering).

In practice, you should match whichever metric the embedding model was trained/optimized with — using cosine similarity on a model trained for dot product (or vice versa) can silently degrade retrieval quality, and this is exactly the kind of detail that separates candidates who've actually shipped a retrieval system from those who haven't.

Q. How do you handle embedding model versioning — what happens when you upgrade to a newer/better embedding model?

ADVANCED

Different embedding models produce vectors in incompatible spaces — you cannot mix vectors from two different models (or even different versions of the same model) in one similarity search; a query embedded with model B will not meaningfully compare against documents embedded with model A.

Upgrading requires a full re-embedding of the entire corpus with the new model, and this is typically done via a blue-green pattern: build a new index with the new model fully in parallel, validate retrieval quality against your eval set, then cut traffic over — rather than mutating the live index in place, which risks a period of inconsistent/degraded search quality.

This is also why embedding model choice is a bigger commitment than it first appears in a design discussion — re-embedding a large corpus has real cost and downtime/consistency implications, and interviewers who've done this in production want to hear that you understand it's not a drop-in swap.

Q. What are Matryoshka embeddings, and why are they useful in production?

ADVANCED

Matryoshka Representation Learning trains embeddings so that meaningful semantic information is concentrated in the leading dimensions of the vector — meaning you can truncate a high-dimensional embedding (e.g., from 1536 down to 256 dimensions) and still retain most of its retrieval quality, rather than needing a separately-trained smaller model.

This gives a single embedding model a tunable storage/speed vs. accuracy tradeoff at query time: use the full-dimension vector when quality matters most, or a truncated version for faster search / lower storage cost on huge corpora, without maintaining multiple models. It's increasingly offered as a built-in option by modern embedding APIs and is a good signal of staying current if you can speak to it.

06

AI Agents & Orchestration

Agent questions are increasingly central to GenAI engineering interviews as products move from single-turn Q&A to multi-step autonomous workflows.

Q. What defines an 'agent' as opposed to a simple LLM call or a fixed pipeline?

FOUNDATIONAL

An agent uses an LLM to dynamically decide its own sequence of actions — which tools to call, in what order, and when to stop — based on intermediate results, rather than following a hardcoded, predetermined sequence of steps.

The defining loop is typically: observe (current state/context) → reason (what should I do next?) → act (call a tool, retrieve information, or respond) → observe the result → repeat until a stopping condition is met. This is what enables agents to handle open-ended tasks a fixed pipeline can't anticipate in advance.

A useful interview framing: a RAG pipeline is a fixed workflow (retrieve, then generate); an agentic RAG system decides for itself whether to retrieve, what to search for, whether the results are sufficient, and whether to search again — the control flow itself is delegated to the model.

Q. How does function/tool calling actually work under the hood?

INTERMEDIATE

The application provides the model with a set of tool definitions (name, description, and a JSON-schema of expected parameters) alongside the prompt. The model is trained/fine-tuned to recognize when a user request maps to one of these tools and to output a structured call — the tool name plus arguments matching the schema — instead of (or alongside) natural language.

Critically, the model does not execute the tool itself — it only generates the structured intent to call it. The application code parses that structured output, actually executes the function/API call, and then feeds the result back into the conversation as a new message so the model can incorporate it into its next response or decide on a next action.

This separation is a key safety property: the model proposes, but the application (and its permission/validation layer) disposes — giving engineers a checkpoint to validate arguments, enforce authorization, or require human approval before anything actually executes.

Q. How do you handle a multi-agent system where agents might disagree, loop, or produce inconsistent results?

ADVANCED

Define clear role boundaries and hand-off protocols up front — e.g., an orchestrator/supervisor agent that routes tasks to specialist sub-agents and owns the final decision, rather than every agent having equal, ambiguous authority. Ambiguous responsibility is the most common source of multi-agent chaos.

Bound the system with hard stopping conditions: max iteration/turn counts, timeouts, and explicit success/failure criteria — without these, reasoning loops (agents repeatedly re-attempting the same failed action) are a very common and costly failure mode.

Use structured, typed hand-offs between agents (not free-form text) wherever possible to reduce miscommunication, and log/trace every step (which agent, what action, what result) so failures are debuggable — multi-agent systems are notoriously hard to debug without strong observability, since a bad outcome could stem from any step in the chain.

Q. How would you evaluate whether an agent is working correctly in production?

ADVANCED

Task-level success rate against a labeled evaluation set of representative tasks (did the agent achieve the intended outcome — not just 'did it respond'), ideally run regularly as a regression suite before shipping prompt or tool changes.

Trajectory/process metrics: number of steps taken (efficiency), tool-call accuracy (did it call the right tool with correct arguments), and whether it recovered gracefully from tool errors — a correct final answer reached through a wasteful or fragile path is still a risk in production.

Safety/guardrail metrics: rate of unauthorized or out-of-scope actions attempted, and how often it correctly asks for human confirmation before high-stakes actions.

In practice this combines automated eval (LLM-as-judge scoring trajectories against a rubric) with human review of a sample of real traces, plus production monitoring for anomalies (unusually long loops, repeated tool failures, cost spikes).

Q. How do you design tool permissions and guardrails so an agent can't take destructive or unauthorized actions?

ADVANCED

Apply least-privilege at the tool level: scope each tool's credentials/API access to only what that specific task needs (e.g., a read-only database tool for a Q&A agent, never a tool with write/delete access unless the task genuinely requires it), rather than giving an agent a single broad, all-purpose credential.

Gate irreversible or high-stakes actions (sending an email, making a purchase, deleting data, modifying production config) behind explicit human confirmation, regardless of how confident the agent's reasoning trace looks — confidence in the reasoning text is not a safety guarantee.

Validate tool arguments server-side before execution (don't trust the model's output blindly even if it matches the schema — e.g., check that a requested file path or account ID is actually one the current user is authorized to touch), and log every tool call with its arguments and result for auditability.

Set hard boundaries the agent cannot reason its way around: rate limits, spend caps, and an explicit denylist of actions/tools that are never available regardless of what the conversation seems to justify.

Q. What's the difference between short-term and long-term memory in an agent system, and how do you implement each?

ADVANCED

Short-term (working) memory is the current conversation/task context — typically just the recent message history and intermediate reasoning/tool results within the active session, held directly in the context window and discarded (or summarized) once the session ends.

Long-term memory persists information across sessions — e.g., user preferences, past decisions, or facts learned in earlier interactions — and is usually implemented outside the context window entirely: stored in a database or vector store, and selectively retrieved (via similarity search or explicit lookup) into the prompt only when relevant to the current turn, rather than kept permanently in context.

A common production pattern is periodic summarization/compaction: as a session grows long, older turns are compressed into a running summary to control context length, while specific durable facts worth keeping indefinitely get written out to long-term storage rather than relying on the summary alone to preserve them.

07

LLMOps, Deployment & Cost

This is where 'can you actually run this in production' gets tested — latency, cost, reliability, and observability.

Q. What levers do you have to reduce LLM inference cost and latency in production?

INTERMEDIATE

Model selection: use the smallest/cheapest model that meets your quality bar for each task rather than one large model for everything — model routing (a lightweight classifier or the LLM itself deciding task difficulty and picking a cheaper model for simple requests) is a widely used cost lever.

Caching: cache exact or semantically-similar repeated queries/responses; prompt caching (reusing the KV cache for a static system prompt or shared context prefix across requests) cuts both cost and time-to-first-token significantly for high-repetition workloads.

Serving optimizations: batching concurrent requests, quantization, speculative decoding (a small draft model proposes several tokens which the large model verifies in parallel, reducing sequential forward passes), and using optimized inference servers (vLLM, TGI) with techniques like PagedAttention for efficient KV-cache memory management.

Prompt/context efficiency: trim unnecessary context, use concise system prompts (or fine-tune to bake in long instructions), and cap max output tokens where a shorter response is acceptable — since both input and output tokens are billed and impact latency.

Q. How do you monitor an LLM application in production? What should trigger an alert?

INTERMEDIATE

Track: latency (p50/p95/p99, and time-to-first-token for streaming UX), error/failure rates (API errors, timeouts, malformed structured outputs), cost per request and daily spend trends, and token usage patterns (input/output ratio, unusually long contexts).

Quality signals: automated groundedness/faithfulness scoring on a sample of live traffic, user feedback signals (thumbs up/down, regeneration rate, session abandonment), and drift detection — is the distribution of inputs or model behavior changing over time relative to your evaluation baseline?

Alerting should be tied to concrete thresholds tied to user/business impact: latency SLA breaches, error rate spikes, cost anomalies (e.g., a prompt-injection loop causing runaway token usage), and sudden drops in user satisfaction signals — not just raw uptime, since an LLM system can be 'up' while silently producing degraded or unsafe outputs.

Q. What is LLM observability/tracing, and why is it different from traditional software observability?

INTERMEDIATE

Traditional observability (logs, metrics, traces) tracks deterministic code paths — the same input reliably produces the same trace. LLM systems are non-deterministic and often involve multi-step chains (retrieval → re-ranking → generation → tool calls → possibly more generation), so a trace needs to capture the full reasoning path: every prompt sent, every intermediate output, every tool call and its result, token counts, and latency at each hop.

This granularity is what makes debugging possible — when an agent produces a wrong answer, you need to see exactly which retrieval returned poor results, which prompt version was used, and what the model actually reasoned through, not just an input/output pair. Tools like LangSmith, Langfuse, and Arize Phoenix exist specifically to capture and visualize these multi-step traces, which off-the-shelf APM tools weren't built for.

Q. How do you safely roll out a change to a prompt or model version in a production LLM application?

ADVANCED

Version and evaluate offline first: run the change against a held-out evaluation set (including known edge cases and past regression examples) and compare quality/cost/latency metrics against the current production version before considering deployment.

Roll out gradually: canary or A/B test the change on a small percentage of live traffic, monitor the same quality/cost/safety signals as full production monitoring, and have an instant rollback path (treat prompts and model versions as deployable artifacts with the same rigor as code).

Keep prompts under version control alongside application code, and maintain a regression suite that runs automatically on every prompt/model change — this catches the common failure mode where a prompt tweak fixes one case but silently breaks several others.

Q. How would you design fallback/failover when your primary LLM provider has an outage or is rate-limiting you?

INTERMEDIATE

Maintain a fallback chain to a secondary provider/model for critical paths, with request routing logic that detects failures (timeouts, 5xx errors, rate-limit responses) and automatically retries against the fallback rather than surfacing an error straight to the user.

This requires keeping prompts reasonably portable across providers (or maintaining provider-specific prompt variants) since models don't behave identically to the same prompt, and testing the fallback path regularly — a failover mechanism that's never been exercised often turns out to be broken exactly when you need it.

For rate limits specifically: implement client-side request queuing/backoff and exponential retry with jitter, and consider tiered degradation (e.g., serve a smaller/faster model or a cached response under load rather than fully failing) so the user experience degrades gracefully instead of breaking outright.

Q. How do you implement streaming responses for an LLM application, and why does it matter for UX?

INTERMEDIATE

Streaming sends generated tokens to the client incrementally as they're produced (typically via server-sent events or a chunked HTTP response) instead of waiting for the full response to complete before returning anything — the client renders tokens as they arrive.

It matters because perceived latency, not just actual total latency, drives user experience: time-to-first-token can be a few hundred milliseconds even if total generation takes several seconds, so streaming makes the application feel dramatically more responsive even though the total time to full completion is unchanged.

Implementation considerations worth naming: handling partial/incomplete structured output mid-stream (you generally can't parse or validate JSON until it's complete, so structured-output and streaming UX need careful design together), and ensuring the client can gracefully handle a stream that's interrupted mid-way (network drop, timeout) without corrupting application state.

08

Evaluation, Safety & Guardrails

Increasingly a dedicated interview section on its own — companies want to know you take output quality and safety seriously, not just capability.

Q. What is LLM-as-judge, and what are its known failure modes?

INTERMEDIATE

LLM-as-judge uses a (typically stronger) LLM to evaluate the outputs of another LLM against a rubric — e.g., scoring helpfulness, correctness, or faithfulness — which scales far better than manual human review and is now standard for offline eval pipelines and even some online quality monitoring.

Known failure modes: position bias (favoring whichever response appears first when comparing two outputs — mitigated by randomizing/swapping order), self-preference bias (a judge model tends to rate outputs from its own model family more favorably), verbosity bias (longer answers often score higher regardless of actual quality), and sensitivity to how the rubric/prompt is worded.

Best practice: validate the judge itself against a smaller human-labeled gold set periodically to confirm it correlates well with human judgment, use clear rubrics with concrete scoring criteria rather than vague quality questions, and don't treat LLM-judge scores as ground truth in isolation — combine with human spot-checks, especially for high-stakes decisions.

Q. What are guardrails, and what are the main categories you'd implement in a production GenAI system?

INTERMEDIATE

Guardrails are checks placed around an LLM's inputs and/or outputs to enforce safety, correctness, and policy constraints beyond what prompting alone reliably guarantees.

Input guardrails: PII detection/redaction, prompt injection/jailbreak detection, topic/scope filtering (rejecting clearly out-of-scope requests before they reach the model), and rate limiting/abuse detection.

Output guardrails: toxicity/harmful content filtering, structured output validation (does the response match the required schema — regenerate or fall back if not), factuality/groundedness checks for RAG-grounded answers, and PII leakage checks before returning a response to the user.

Architecturally, these are usually implemented as separate lightweight classifiers or rule-based checks that run before/after the main LLM call — not baked into the main model's prompt alone — because a single prompt asking the model to 'also be careful about X' is much less reliable than a dedicated, independently-tested check.

Q. How do you handle PII in a GenAI application that processes user data?

INTERMEDIATE

Detect and redact/mask PII (names, emails, SSNs, phone numbers, etc.) before it reaches the LLM where possible, especially if using a third-party API where you don't control data retention — using either regex/NER-based detectors or a dedicated PII-detection service.

If PII must be processed (e.g., a support bot legitimately needs a customer's name), scope data access tightly, ensure it's not logged in plaintext in traces/observability tools, and confirm the LLM provider's data usage/retention policy matches your compliance requirements (many enterprise API tiers offer zero data retention).

For RAG systems indexing internal documents, apply the same access-control logic to retrieval as you would to the underlying data — i.e., retrieval should respect the same document-level permissions as your source systems, or you risk a user's query surfacing data they weren't authorized to see in the first place.

Q. What's the difference between hallucination and a factually correct but ungrounded/unsupported answer — and why does the distinction matter for evaluation?

ADVANCED

A hallucination is a claim the model states confidently that is actually false. An ungrounded answer might be factually true but wasn't actually derivable from the retrieved context provided — it came from the model's parametric memory instead, which is a subtler and often overlooked failure mode in RAG evaluation.

The distinction matters because a RAG system is supposed to answer from the provided sources — an ungrounded-but-true answer still breaks the trust/citation contract of the system (the citation may be wrong or missing, and if the underlying source is later found to be false or outdated, users have no way to know the model wasn't actually relying on it). Faithfulness evaluation specifically measures groundedness against the retrieved context, independent of whether the claim happens to be true in the real world — this is why faithfulness and correctness are tracked as separate metrics in rigorous RAG evaluation.

Q. What does red-teaming an LLM application actually involve, and how is it different from standard QA testing?

ADVANCED

Standard QA tests whether the system works correctly on expected inputs. Red-teaming deliberately searches for inputs that make the system fail or misbehave — adversarial prompts designed to elicit jailbreaks, harmful content, prompt injection success, PII leakage, or policy violations — treating the system as a target to break rather than a spec to confirm.

It combines manual adversarial testing by people specifically trained to think like an attacker, automated adversarial prompt generation/fuzzing, and testing across known jailbreak patterns and edge cases specific to your domain (e.g., a financial app should be red-teamed for attempts to extract other users' data or get unauthorized financial advice framed as 'hypothetical').

The output of red-teaming should directly feed back into guardrails, system prompt hardening, and the evaluation suite — every successful red-team finding should become a permanent regression test, so the same class of failure is automatically caught before it can ship again.

Q. How do you evaluate an LLM application for bias or fairness issues?

ADVANCED

Build evaluation sets that specifically probe for differential treatment across demographic groups or protected attributes relevant to your application — e.g., testing whether a resume-screening assistant's outputs shift systematically based on names/genders/schools associated with different demographics, holding the actual qualifications constant.

Use both automated metrics (statistical parity checks across groups on quantifiable outcomes) and structured human review, since many fairness issues are subtle enough that a single aggregate accuracy number won't surface them — a model can have excellent average performance while systematically underperforming for a specific subgroup.

Be candid in an interview that bias evaluation is an ongoing process, not a one-time checkbox: it should be part of the regression suite for every model/prompt update, since a change that fixes one failure mode can easily introduce or worsen another one that wasn't being actively tested.

09

Classical ML & Deep Learning Fundamentals

Even GenAI-focused loops often open with core ML/DL fundamentals to confirm the basics are solid before going deep on transformers. Don't skip this section assuming it's 'too basic' — a shaky answer here is a fast, avoidable way to lose credibility early in the interview.

Q. Explain the bias-variance tradeoff. How do you diagnose whether a model is overfitting or underfitting?

FOUNDATIONAL

Bias is error from overly simplistic assumptions (the model can't capture the true pattern even with unlimited data); variance is error from being overly sensitive to the specific training set (the model captures noise, not just signal). Total generalization error decomposes roughly into bias² + variance + irreducible noise, and the two typically trade off against each other as model complexity changes.

Underfitting (high bias) shows up as poor performance on both training and validation/test data — the model is too simple to fit even the training set well. Overfitting (high variance) shows up as strong training performance but a large, widening gap to validation/test performance — the model has memorized training-specific noise instead of learning generalizable patterns.

Diagnosis in practice: plot training vs. validation loss curves over training steps/epochs — a widening gap is the classic overfitting signature. Fixes for overfitting include more data, regularization (dropout, weight decay, early stopping), or a simpler model; fixes for underfitting include a more expressive model, more training time, or better features.

Q. What is backpropagation, and how do optimizers like SGD, Momentum, and Adam differ?

FOUNDATIONAL

Backpropagation computes the gradient of the loss with respect to every parameter in the network by applying the chain rule backward through the computational graph, layer by layer — this is what makes training deep networks with millions/billions of parameters tractable, since it computes all gradients in a single backward pass rather than one at a time.

Plain SGD updates each parameter directly opposite its gradient, scaled by a learning rate — simple, but can be slow and oscillate in narrow/curved loss landscapes. Momentum adds a running average of past gradients to the update, which accelerates progress in consistent directions and dampens oscillation, much like a ball rolling downhill picking up speed.

Adam combines momentum with an adaptive, per-parameter learning rate derived from a running average of squared gradients — parameters with consistently large gradients get smaller effective steps and vice versa. This adaptivity is why Adam (and variants like AdamW) is the default optimizer for training most modern deep learning models, including LLMs, despite plain SGD sometimes generalizing slightly better with careful tuning.

Q. What's the difference between cross-entropy loss and mean squared error (MSE), and when would you use each?

FOUNDATIONAL

Cross-entropy loss measures the divergence between a predicted probability distribution and the true (one-hot or soft) label distribution — it's the standard choice for classification tasks, including next-token prediction in language models, where the output is a probability distribution over a discrete set of classes/tokens.

MSE measures the squared difference between predicted and true continuous values — it's the standard choice for regression tasks where the output is a real number (e.g., predicting a price or a continuous score), and it penalizes large errors disproportionately more than small ones due to the squaring.

Using MSE for a classification task (or cross-entropy for regression) generally trains poorly — MSE's gradient behavior doesn't push probability outputs toward confident correct predictions as effectively as cross-entropy does, which is why the loss function needs to match the actual structure of the prediction task, not just be picked by convention.

Q. What's the difference between batch normalization and layer normalization, and why do Transformers use LayerNorm specifically?

INTERMEDIATE

Batch normalization normalizes each feature across the batch dimension — it computes mean/variance for a given feature over all examples in the current batch. This makes it dependent on batch size and batch composition, and it behaves differently at train time (batch statistics) vs. inference time (running statistics), which adds complexity.

Layer normalization instead normalizes across the feature dimension for each individual example independently, regardless of batch size — every token's representation is normalized using only its own features, not other examples in the batch.

Transformers use LayerNorm because sequence lengths and batch compositions vary a lot in NLP, making batch statistics unstable and less meaningful per-token; LayerNorm's per-example independence is a much better fit for variable-length sequence data and doesn't create a train/inference mismatch, which matters a great deal for autoregressive generation where inference happens one token at a time.

Q. What are vanishing and exploding gradients, and how are they mitigated in deep networks?

INTERMEDIATE

During backpropagation, gradients are computed by repeatedly multiplying through many layers via the chain rule. If those per-layer factors are consistently less than 1, the gradient shrinks exponentially as it propagates backward through many layers (vanishing), making early layers barely update. If the factors are consistently greater than 1, the gradient grows exponentially (exploding), causing unstable, huge parameter updates.

Mitigations: residual/skip connections (used throughout Transformers) give gradients a direct, unimpeded path backward, which is one of the main reasons very deep networks became trainable at all; careful weight initialization (e.g., Xavier/He initialization) keeps activations and gradients in a reasonable range from the start; normalization layers (LayerNorm/BatchNorm) keep activation magnitudes stable layer to layer; and gradient clipping caps the gradient norm to directly prevent exploding updates, which is standard practice in LLM training.

Q. Precision, recall, F1, and ROC-AUC — when does each metric matter most?

INTERMEDIATE

Precision = (true positives) / (predicted positives) — of everything the model flagged as positive, how much was actually correct. Matters most when false positives are costly (e.g., a fraud-flagging system that blocks legitimate transactions too often erodes user trust).

Recall = (true positives) / (actual positives) — of everything that was actually positive, how much did the model catch. Matters most when false negatives are costly (e.g., a content-moderation classifier that misses genuinely harmful content is a worse failure than an occasional over-flag).

F1 is the harmonic mean of precision and recall, useful as a single balanced number when both error types matter and especially when classes are imbalanced, since plain accuracy can look deceptively high on an imbalanced dataset by just predicting the majority class.

ROC-AUC measures ranking quality across all possible classification thresholds — how well the model separates positive from negative examples regardless of where you set the cutoff. It's useful for comparing models independent of a specific operating threshold, but can be misleadingly optimistic on heavily imbalanced datasets, where precision-recall AUC is usually the more informative choice.

10

Multimodal Models & Diffusion

As GenAI products increasingly span text, image, audio, and video, expect at least a couple of questions probing whether you understand what's structurally different about non-text modalities — even in a primarily text/LLM-focused role.

Q. How do vision-language models like CLIP and LLaVA actually work?

INTERMEDIATE

CLIP trains an image encoder and a text encoder jointly using contrastive learning: given a batch of (image, caption) pairs, it pulls the embeddings of matching pairs closer together and pushes non-matching pairs apart, so that at the end both modalities share a common embedding space where semantically similar images and text land near each other — with no generative objective at all, just alignment.

LLaVA-style models instead connect a pretrained vision encoder to a pretrained LLM: an image is split into patches, embedded by a vision transformer (ViT) encoder, projected through a small trainable adapter/projection layer into the same embedding dimension as the LLM's token embeddings, and then fed into the LLM alongside text tokens — the model is then fine-tuned so it learns to jointly attend to image-patch embeddings and text tokens as one unified sequence.

The key distinction to articulate in an interview: CLIP is fundamentally about alignment/retrieval (are this image and this text related?), while LLaVA-style architectures are about generation (produce text conditioned on both image and text input) — different objectives, and often used together in a pipeline (CLIP-style encoders are frequently the vision backbone feeding into a LLaVA-style generative model).

Q. Walk through how diffusion models generate images — the forward and reverse process.

INTERMEDIATE

The forward process is fixed and not learned: starting from a real image, Gaussian noise is added incrementally over many steps until the image becomes pure noise. This defines a known, tractable path from data to noise.

The reverse process is what the model learns: a neural network (typically a U-Net, or a Transformer-based DiT in newer models) is trained to predict and remove a small amount of noise at each step, gradually denoising a random noise sample back into a coherent image over many iterations.

Generation at inference time starts from pure random noise and repeatedly applies the learned denoising step, guided by a conditioning signal (e.g., a text prompt embedding, typically via cross-attention layers injected into the denoising network) — sampling a genuinely new image rather than reconstructing a specific training example.

Q. What's the difference between diffusion models and GANs for image generation?

ADVANCED

GANs (Generative Adversarial Networks) train two networks adversarially: a generator that tries to produce realistic images from random noise in a single forward pass, and a discriminator that tries to distinguish generated images from real ones — the generator improves by learning to fool an increasingly capable discriminator.

Diffusion models instead learn to iteratively denoise, requiring many sequential forward passes to generate one image (though modern techniques have cut this down significantly), rather than GANs' single-pass generation.

In practice: GANs are much faster at inference (one pass) but are notoriously unstable to train (mode collapse, adversarial training dynamics that don't converge cleanly) and tend to produce less diverse outputs. Diffusion models are slower per generation but train more stably, scale better with data/compute, and currently produce higher-fidelity, more diverse results — which is why diffusion architectures (Stable Diffusion, DALL-E, Midjourney's underlying approach) have become the dominant paradigm for state-of-the-art image generation, with GANs now more common in specialized, latency-sensitive use cases.

Q. How does a text-to-image pipeline like Stable Diffusion actually work end-to-end?

ADVANCED

A text encoder (typically a CLIP text encoder) converts the input prompt into an embedding that captures its semantic content — this embedding will condition every step of the generation process.

Rather than diffusing in raw pixel space (computationally very expensive for high-resolution images), latent diffusion models first compress the image into a smaller latent representation using a pretrained autoencoder (a VAE), and run the entire diffusion/denoising process in this much lower-dimensional latent space — this is the key efficiency insight that made high-resolution diffusion models practical to train and run.

A U-Net (or newer DiT/Transformer-based denoiser) then iteratively denoises a random latent, at each step cross-attending to the text embedding so the denoising direction is guided by the prompt, gradually shaping random noise into a latent that represents a coherent image.

Finally, the VAE decoder converts the denoised latent back into a full-resolution pixel-space image. Being able to name this three-part structure — text encoder, latent-space diffusion U-Net, VAE decoder — is usually what separates a candidate who has actually read the architecture from one who's only used the product.

Q. How are images actually tokenized or represented as input to a multimodal LLM?

INTERMEDIATE

Unlike text, which has a natural discrete vocabulary, images are continuous pixel grids — so the standard approach (following Vision Transformer/ViT) is to split the image into fixed-size patches (e.g., 16×16 pixels), flatten each patch, and linearly project it into an embedding vector — conceptually the image equivalent of a 'token,' though it represents a patch of pixels rather than a discrete symbol.

These patch embeddings, plus positional information indicating where each patch sits in the image, are then projected (often through a small trainable adapter) into the same embedding space as the LLM's text token embeddings, so the model's self-attention can operate over a single unified sequence mixing image-patch tokens and text tokens.

Some newer approaches use a learned tokenizer that maps image regions to a discrete codebook (closer to true 'image tokens' analogous to text tokens, e.g., VQ-VAE-style approaches), which is worth mentioning if the conversation goes deeper — but the patch-embedding-via-ViT approach is the more common default in today's production multimodal LLMs.

Q. How do you evaluate the quality of generated images?

ADVANCED

FID (Fréchet Inception Distance) compares the statistical distribution of features (extracted by a pretrained classifier network) between a set of generated images and a set of real images — lower FID means the generated distribution is closer to real image statistics, and it's the most widely used automated metric for overall generation quality/realism.

CLIP score measures how well a generated image matches its text prompt, by embedding both with CLIP and computing their similarity — this specifically evaluates prompt-adherence/alignment, which FID does not capture at all (an image can look photorealistic yet completely ignore the prompt).

Neither metric fully captures human aesthetic judgment or subtle failure modes (extra fingers, broken text, physically implausible objects), so rigorous evaluation still pairs these automated metrics with structured human evaluation — pairwise comparisons or rubric-based scoring by human raters — especially before shipping a user-facing generation feature.

11

Infrastructure & Deployment Engineering

The LLMOps section covered the what and why of production cost, latency, and monitoring; this section covers the how — the actual infrastructure decisions behind serving models reliably at scale.

Q. How would you containerize and deploy an LLM inference service? What goes into the Docker image?

INTERMEDIATE

A production inference image typically includes: the serving framework (e.g., vLLM, TGI, or a custom FastAPI wrapper around the model), pinned dependency versions (critical for reproducibility — GPU/CUDA driver mismatches are one of the most common 'works on my machine' failures in ML infra), and the model weights themselves — though for large models, weights are often mounted from external storage or pulled at startup rather than baked into the image, to keep image size and build times manageable.

The base image should match the CUDA/driver version the serving framework was built and tested against, since GPU inference is far more sensitive to environment mismatches than typical stateless web services. Health-check and readiness-probe endpoints matter more than usual here too, since a model that's still loading weights into GPU memory needs to report 'not ready' rather than accept and fail requests during startup.

Q. How do you use Kubernetes to scale a GPU-backed inference service, and what's different from scaling stateless web services?

ADVANCED

GPUs are a scarce, expensive, and non-fungible resource compared to CPU/memory — Kubernetes needs GPU device plugins to schedule pods onto nodes with available GPUs, and autoscaling has to account for the fact that spinning up a new GPU node (and loading a large model's weights onto it) can take minutes, not seconds, unlike a typical stateless web pod.

Standard CPU-utilization-based autoscaling often doesn't reflect real load for LLM serving — request queue depth, GPU memory utilization, or custom metrics like tokens-per-second throughput are usually better autoscaling signals than CPU percentage.

Because cold-start latency is so much higher for GPU workloads, teams often keep a warm minimum replica count rather than scaling to zero, and use request queuing/batching at the serving layer to absorb bursts without needing to scale hardware instantly. Model sharding across multiple GPUs (for models too large for one GPU) adds further complexity — Kubernetes needs to schedule sharded replicas as a coordinated group, not independent pods.

Q. Serverless GPU inference vs. always-on dedicated servers — how do you decide?

INTERMEDIATE

Serverless GPU (pay-per-request, scales to zero) fits bursty, unpredictable, or low-volume traffic well — you avoid paying for idle GPU time, at the cost of cold-start latency when a new instance has to spin up and load model weights, which can be seconds to tens of seconds for large models.

Always-on dedicated servers fit steady, predictable, high-volume traffic — you pay for capacity whether or not it's fully utilized, but you eliminate cold-start latency entirely and get more predictable performance, which matters a lot for latency-sensitive user-facing features.

A common middle ground: a small warm pool of always-on capacity to absorb baseline traffic with zero cold-start, with serverless or autoscaled capacity layered on top to absorb spikes — the decision should be driven by your actual traffic shape (steady vs. bursty) and how latency-sensitive the specific feature is, not a blanket policy across every service.

Q. What is continuous batching, and why does it matter for LLM serving throughput?

ADVANCED

Naive (static) batching groups a fixed set of requests together and waits for every request in the batch to finish generating before processing the next batch — but LLM requests finish at very different times (short answers finish fast, long answers take much longer), so static batching wastes significant GPU capacity idling on requests that finished early while waiting for the slowest one.

Continuous (dynamic) batching instead adds new incoming requests into the running batch as soon as a slot frees up (i.e., as soon as any request in the batch finishes), keeping the GPU consistently saturated with useful work rather than idling — this is one of the single biggest throughput improvements in modern LLM serving frameworks (vLLM, TGI) compared to naive batching, often multiplying achievable throughput several times over on the same hardware.

Q. What's the difference between tensor parallelism and pipeline parallelism for serving very large models across multiple GPUs?

ADVANCED

Tensor parallelism splits individual weight matrices/layers across multiple GPUs — e.g., a single attention or feed-forward layer's computation is divided across GPUs, which must communicate (all-reduce) after every split operation. This requires very fast interconnects (like NVLink) between GPUs since communication happens constantly within a single forward pass, but it keeps latency low since all GPUs work on the same request simultaneously.

Pipeline parallelism instead assigns different full layers to different GPUs (GPU 1 handles layers 1–10, GPU 2 handles layers 11–20, etc.), passing activations between GPUs sequentially as data flows through the layers — this needs far less inter-GPU communication bandwidth than tensor parallelism, but can leave GPUs idle waiting for the previous stage (a 'pipeline bubble') unless carefully scheduled with overlapping micro-batches.

In practice, serving very large models often combines both: tensor parallelism within a node (where GPUs have fast NVLink interconnects) and pipeline parallelism across nodes (where inter-node bandwidth is more limited) — knowing when and why to combine them is a strong signal of real infrastructure experience.

Q. How do you design load balancing and routing across multiple model replicas or multiple providers?

INTERMEDIATE

Within a single model's replicas: route based on actual load signals (queue depth, current KV-cache/memory utilization) rather than simple round-robin, since LLM requests have wildly varying costs (a 50-token vs. 4000-token generation) that round-robin doesn't account for — least-loaded routing generally outperforms round-robin for this reason.

Across multiple providers/models (e.g., a primary and fallback provider, or routing between a large and small model by task difficulty): implement health-check-based failover so a struggling provider is automatically routed around, and consider weighted routing to gradually shift traffic during a migration or A/B test rather than an abrupt full cutover.

Sticky sessions can matter for multi-turn conversations if server-side state (like an uncommitted KV cache for a long-running session) is tied to a specific replica — otherwise, requests should generally be treated as stateless and freely routable to whichever replica has capacity.

12

Coding & Practical Exercises

Many GenAI engineer loops still include a live or take-home coding round. These exercises cover the building blocks that come up repeatedly — retrieval, chunking, resilience patterns, and agent control flow — with the reasoning an interviewer is actually listening for, not just a working solution.

CODING EXERCISE

Implement cosine similarity search over a list of embeddings.

This is the retrieval building block behind almost every RAG system. The key things an interviewer is checking: do you normalize vectors correctly, do you avoid an $O(n^2)$ mistake, and can you reason about doing this at scale (this naive version is O(n) per query — mention that production systems replace this linear scan with an ANN index like HNSW once the corpus grows past a few thousand vectors).

```
import numpy as np

def cosine_similarity(query: np.ndarray, corpus: np.ndarray) -> np.ndarray:
    """query: shape (d,)   corpus: shape (n, d)   returns: shape (n,)"""
    query_norm = query / (np.linalg.norm(query) + 1e-10)
    corpus_norm = corpus / (np.linalg.norm(corpus, axis=1, keepdims=True) + 1e-10)
    return corpus_norm @ query_norm

def top_k(query: np.ndarray, corpus: np.ndarray, k: int = 5) -> list[int]:
    scores = cosine_similarity(query, corpus)
    return list(np.argsort(-scores)[:k])
```

CODING EXERCISE

Write a text-chunking function with configurable chunk size and overlap.

Interviewers are checking whether you handle the overlap logic correctly (a common off-by-one source of bugs), whether you guard against an infinite loop when overlap ≥ chunk size, and whether you're thinking in tokens vs. characters (this sketch chunks by characters for simplicity — mention you'd swap in a tokenizer's encode/decode for production accuracy).

```
def chunk_text(text: str, chunk_size: int = 500, overlap: int = 50) -> list[str]:
    if overlap >= chunk_size:
        raise ValueError("overlap must be smaller than chunk_size")
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return chunks
```

CODING EXERCISE

Implement an exponential backoff retry wrapper for a flaky LLM API call.

This tests whether you know the real pattern (exponential growth + jitter to avoid a thundering herd of synchronized retries) versus a naive fixed-delay retry loop, and whether you distinguish retryable errors (rate limits, timeouts, 5xx) from non-retryable ones (a 400 bad request should fail fast, not retry).

```
import time, random

def call_with_backoff(fn, max_retries: int = 5, base_delay: float = 1.0):
    for attempt in range(max_retries):
        try:
            return fn()
        except RetryableError:
            if attempt == max_retries - 1:
                raise
            delay = base_delay * (2 ** attempt) + random.uniform(0, 0.5)
            time.sleep(delay)
```

CODING EXERCISE

Implement a simple LRU cache for caching prompt/response pairs.

This checks basic data-structure fluency (O(1) get/put using a hash map + doubly linked list, or Python's OrderedDict as a shortcut) and whether you think about what the cache key actually should be for an LLM call — usually a hash of the full prompt plus relevant parameters (model, temperature), not just the raw user message.

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int = 1000):
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key: str):
        if key not in self.cache:
            return None
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key: str, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)
```

CODING EXERCISE

Write a token-bucket rate limiter for outgoing API calls.

This is checking whether you understand token-bucket vs. fixed-window rate limiting (token bucket allows brief bursts up to the bucket size while still enforcing a long-run average rate, which better matches how most LLM provider rate limits actually behave) and whether your refill logic is time-based and correct rather than a hacky counter reset.

```
import time

class TokenBucket:
    def __init__(self, rate: float, capacity: int):
        self.rate = rate          # tokens added per second
        self.capacity = capacity
        self.tokens = capacity
        self.last_refill = time.monotonic()

    def _refill(self):
        now = time.monotonic()
        elapsed = now - self.last_refill
        self.tokens = min(self.capacity, self.tokens + elapsed * self.rate)
        self.last_refill = now

    def allow(self) -> bool:
        self._refill()
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False
```

CODING EXERCISE

Sketch the core loop of a ReAct-style agent with tool calling.

This is a systems-design-through-code question — interviewers want to see the reason → act → observe loop structured cleanly, an explicit max-iteration cap (a very common thing candidates forget), and a clear separation between 'the model decided to call a tool' and 'the application actually executes it,' which is the safety-relevant boundary discussed in the Agents section.

```
def run_agent(user_input: str, tools: dict, max_steps: int = 8) -> str:
    messages = [{"role": "user", "content": user_input}]

    for step in range(max_steps):
        response = llm_call(messages, tools=list(tools.keys()))

        if response.tool_call is None:
            return response.content  # model produced a final answer

        tool_name = response.tool_call.name
        tool_args = response.tool_call.arguments
        if tool_name not in tools:
            messages.append({"role": "tool", "content": f"Unknown tool: {tool_name}"})
            continue

        result = tools[tool_name](**tool_args)   # application executes, not the model
        messages.append({"role": "assistant", "tool_call": response.tool_call})
        messages.append({"role": "tool", "content": str(result)})

    return "Max steps reached without a final answer."
```

13

Scenario-Based Questions

These simulate the "here's a mess, walk me through your triage" questions that senior GenAI interviews increasingly favor over textbook recall. Interviewers are scoring your diagnostic process — where you look first, how you isolate root cause, and what structural fix you'd ship — as much as the final answer.

SCENARIO

Users report that the chatbot gives noticeably different answers to the exact same question when asked minutes apart. How do you debug this?

- First isolate the variable: check temperature/sampling settings (T > 0 will naturally produce different phrasing and sometimes different substance run to run) versus an actual retrieval instability (is the RAG pipeline returning different chunks for the same query, e.g., due to a recently updated index or a non-deterministic re-ranker).
- Check for a silent model or prompt version change — an unpinned model alias, an auto-updated provider model, or an A/B test the requester wasn't told about are common culprits that look like 'random' inconsistency but are actually deterministic version drift.
- If it's genuinely sampling variance and consistency matters for this use case, lower temperature toward 0 for factual/deterministic tasks, or add a caching layer for repeated identical queries.
- Add this exact case to your regression eval set once resolved, and consider logging model/prompt version alongside every response so 'it gave a different answer' is always trivially explainable after the fact.

SCENARIO

Production latency for your LLM feature suddenly tripled with no code deploy and no traffic spike. Walk through your triage.

- Check provider-side status first (is this an upstream outage/degradation, not your system) — provider dashboards and status pages before assuming the problem is internal.
- Look at token counts: has average input or output length grown (e.g., a retrieval change now returns more/larger chunks, or conversation histories have grown longer than usual) — latency scales with token count, so a silent increase in context size is a very common root cause.
- Check for a model routing change (did a fallback path kick in to a slower model), a KV-cache/batching regression on self-hosted infra, or a downstream dependency (vector DB, re-ranker service) that got slow and is now the bottleneck in the chain rather than the LLM call itself.
- Isolate with tracing: break down end-to-end latency by stage (retrieval, re-ranking, generation, post-processing) rather than treating 'the LLM call' as one black box — this is exactly what LLM observability tooling is for.

SCENARIO

Your RAG system starts confidently citing information from a document that was deleted from the source system last week. What happened, and how do you fix it?

- Root cause: the vector index wasn't updated when the source document was deleted — a very common gap when ingestion pipelines handle 'add new document' well but don't have a matching deletion/sync path.
- Immediate fix: run a reconciliation job to remove orphaned vectors for deleted source documents, and audit whether other deletions/updates have similarly gone unsynced.
- Structural fix: the ingestion pipeline needs to treat the vector index as a derived, kept-in-sync view of the source system, not a one-way write target — implement change-data-capture or a scheduled diff/reconciliation process, not just an 'add on create' webhook.
- Add a freshness check to your monitoring: alert if a source system's document count and the index's document count drift apart beyond a threshold, so this class of staleness is caught automatically rather than by a confused user.

SCENARIO

A model you fine-tuned scores well on your held-out evaluation set but performs noticeably worse with real users. What are the likely causes?

- Distribution mismatch: your eval set may not represent real traffic — check whether real user queries are more varied, longer, more ambiguous, or cover edge cases the training/eval data didn't include.
- Overfitting to the eval set itself, especially if it was reused many times during iterative prompt/fine-tuning development — the eval set effectively stopped being 'held out' in practice.
- A mismatch between the eval metric and what actually matters to users — e.g., high exact-match accuracy on a narrow task doesn't capture tone, helpfulness, or handling of the messy, underspecified requests real users actually send.
- Fix: sample real production traces (with user consent/privacy handling as needed), have them reviewed, and use them to both diagnose the gap and expand the eval set going forward — the fix is almost always 'make the eval set look more like reality,' not just 'train more.'

SCENARIO

An internal agent with database and email tools gets manipulated via a crafted user message into attempting to email out data it shouldn't. The action was blocked by a permission check, but it was close. What changes do you make?

- Treat this as confirmation that prompt-level instructions alone are not a security boundary — the permission check that caught it is the actual control, and that's the layer to invest in further, not just better wording in the system prompt.
- Audit tool scopes: does the agent actually need broad email-send capability, or can it be scoped to specific pre-approved recipients/templates for this use case, shrinking the blast radius of a future successful manipulation.
- Add this exact prompt pattern to a red-team regression suite so the specific manipulation (and variations of it) are tested automatically before every future deployment.
- Consider adding a human-confirmation step for any outbound communication action regardless of how the agent got there, since 'blocked this time' is not the same guarantee as 'will always be blocked.'

SCENARIO

Monthly LLM API spend has tripled over three months while user traffic has stayed roughly flat. How do you investigate and fix it?

- Break spend down by endpoint/feature and by token type (input vs. output) — a single feature or a single growing conversation-history pattern is often responsible for the bulk of the increase, not a broad uniform rise.
- Check for context growth: are conversation histories, RAG context sizes, or few-shot examples silently growing over time (e.g., an unbounded conversation history being replayed in full on every turn)?
- Check for retry/loop inflation: a bug causing duplicate calls, an agent looping more than expected, or a fallback path being hit far more often than intended can multiply cost without a visible traffic change.
- Once diagnosed, apply targeted fixes: cap/truncate or summarize long histories, add prompt caching for repeated static context, and consider model routing so only genuinely hard requests use the most expensive model.

SCENARIO

Vector search returns irrelevant results specifically for short, acronym-heavy technical queries, while longer natural-language queries work fine. Diagnose it.

- This is a classic dense-retrieval weakness: embedding models often struggle with short queries and rare acronyms/proper nouns because there's little semantic context for the model to embed meaningfully, and the acronym may barely appear in the training data of the embedding model.
- Check whether hybrid search (adding BM25/keyword matching) is enabled — sparse retrieval typically handles exact acronym/term matching far better than dense embeddings alone, and this symptom pattern is almost a textbook case for why hybrid search exists.
- Consider query expansion for short queries (e.g., using the LLM itself to expand an acronym or rephrase a terse query into a fuller semantic query before embedding it) as a complementary fix.
- Validate with a targeted eval slice: build a small test set specifically of short/acronym queries and measure retrieval metrics on that slice separately, since it can look fine in aggregate metrics while being broken for this specific query type.

SCENARIO

Leadership wants to switch from a large frontier model to a smaller, cheaper model to cut costs, without hurting quality. How do you approach this migration?

- Before touching anything: build (or confirm you already have) a solid evaluation set representative of real production traffic, with the current model's outputs as a quality baseline to compare against — you can't validate a migration without one.
- Run the candidate smaller model against the same eval set and compare quality metrics, not just a handful of spot checks, and pay particular attention to the hardest/most complex examples in the set, since smaller models typically degrade unevenly — often fine on easy queries, noticeably worse on complex ones.
- Consider a hybrid approach rather than an all-or-nothing swap: route simple/common requests to the smaller model and keep the larger model for complex or high-stakes requests, using a lightweight classifier or confidence signal to decide.
- Roll out gradually with canary traffic and live quality monitoring (not just offline eval) before a full cutover, with an easy rollback path if production quality signals (user feedback, escalation rate) degrade.

SCENARIO

An agent occasionally gets stuck calling the same tool repeatedly in a loop for certain inputs, burning cost and never returning an answer. How do you fix it?

- Add a hard iteration/step cap and timeout as an immediate safety net — every agent loop needs an explicit stopping condition that doesn't depend on the model 'deciding' to stop.
- Investigate the specific trigger: often the tool is returning an error or an unexpected result format the model doesn't know how to handle, so it keeps retrying the same action hoping for a different result — inspect the actual tool outputs in the failing traces, not just the agent's reasoning text.
- Improve the tool's error messages to be actionable (tell the model specifically why the call failed and what to try differently) rather than a generic failure, since a vague error is a common cause of the model retrying blindly instead of adapting.
- Add loop detection: if the same tool+arguments combination repeats within a session, break out explicitly and either try an alternate strategy or return a clear 'unable to complete this' response instead of continuing silently.

SCENARIO

You discover that employees have been pasting sensitive internal data into prompts sent to a third-party LLM API. What do you do?

- Immediate: assess the provider's actual data retention/training policy for the tier being used (many enterprise API tiers offer zero data retention — confirm which applies here) to understand real exposure, then flag to security/compliance regardless of the answer.
- Add input-side PII/sensitive-data detection and redaction before requests leave your systems, rather than relying on user discipline alone — automated technical controls consistently outperform policy-only guidance for this kind of risk.
- Provide a sanctioned, properly-configured alternative (e.g., an internal gateway with the right data-handling agreement in place) so there's a compliant path that's actually as easy to use as the risky one — pure restriction without an alternative tends to just push the behavior further underground.
- Follow up with clear policy communication and training, but treat this primarily as a systems/tooling gap to close, not just a behavior problem to lecture about.

14

System Design Case Studies

Senior GenAI engineer interviews almost always include one open-ended design question. These three walkthroughs model the structure and depth expected: ingestion/retrieval architecture, safety guardrails, and evaluation strategy — not just a model name.

Design a customer-support chatbot grounded in a company's help-center documentation

- Ingestion: scrape/sync help-center articles, chunk by section headers, embed with a domain-appropriate model, index in a hybrid (dense + BM25) vector store; set up a re-indexing pipeline to catch article updates.
- Retrieval: hybrid search + cross-encoder re-ranking, top-3–5 chunks passed to the LLM with citations required in the output.
- Guardrails: scope classifier to reject off-topic questions, PII redaction on user input, faithfulness check before returning an answer, and a confident 'I don't know, let me connect you to a human' fallback when retrieval confidence is low.
- Escalation: clear handoff to a human agent with full conversation context when the bot is uncertain or the user explicitly asks for a human.
- Evaluation: golden set of real support tickets with correct answers, tracking faithfulness, resolution rate, and escalation rate over time; online A/B testing before rolling out prompt/model changes.

Design a code-generation copilot for an internal engineering team

- Context assembly: retrieve relevant code from the repo (via embeddings over code chunks + symbol/dependency graph awareness), include open files, recent edits, and relevant style/lint config in the prompt.
- Model choice: a code-specialized model (or a general model fine-tuned/prompted with few-shot examples of the team's code style); consider a smaller, faster model for inline autocomplete vs. a larger model for chat-based multi-file tasks.
- Safety: never auto-execute generated code without review; static analysis / lint / test-run as a validation gate before suggesting a change is 'ready.'
- Latency: autocomplete needs sub-200ms time-to-first-token — this pushes toward smaller models, aggressive caching of repo context, and streaming generation.
- Evaluation: acceptance rate of suggestions, downstream bug rate in accepted suggestions, and developer time-saved surveys — not just code-generation benchmark scores in isolation.

Design a system to summarize and extract structured data from thousands of long legal/financial documents

- Chunking: structure-aware chunking respecting section/clause boundaries; consider a map-reduce summarization pattern for documents exceeding context window (summarize chunks, then summarize the summaries) or a long-context model if quality on very long documents is critical.
- Structured extraction: define a strict JSON schema for required fields, use function-calling/structured-output mode rather than free-text parsing, and validate output against the schema with automatic retry on failure.
- Accuracy: for high-stakes fields (dollar amounts, dates, party names), pair LLM extraction with a verification pass — e.g., regex/rule-based cross-check, or a second LLM call specifically verifying each extracted field is directly quoted/traceable to the source text.
- Human-in-the-loop: route low-confidence extractions to human review rather than silently guessing, given the cost of an error in a legal/financial context.
- Cost: batch processing with a cheaper model where accuracy allows, reserving the most capable (and expensive) model for the highest-stakes documents or fields.

15

Project & Portfolio Questions

These questions ask you to narrate your own work. The difference between a strong and weak answer is almost never the project itself — it's whether you can name specific decisions, tradeoffs, failures, and metrics instead of describing a generic pipeline.

PROJECT & PORTFOLIO

Walk me through a RAG system you built, end to end.

Structure the answer around the pipeline, not just the outcome: what the corpus was and why chunking/embedding choices fit it, what retrieval strategy you used and why (dense-only vs. hybrid, re-ranking or not), how you evaluated quality before shipping (name real metrics — faithfulness, retrieval precision/recall — not just 'it worked well'), and at least one concrete problem you hit and how you diagnosed and fixed it. Interviewers are listening for whether you actually made and can defend specific design decisions, versus describing a generic RAG tutorial.

PROJECT & PORTFOLIO

Describe a time you had to significantly change your chunking or retrieval strategy after initial results were disappointing. What triggered it, and what did you change?

This question is testing your debugging process as much as the final fix. Strong answers name the specific symptom (e.g., 'answers were technically correct but missing context' or 'retrieval kept surfacing near-duplicate chunks'), how you traced it back to a root cause (was it chunk size, embedding model mismatch, missing hybrid search, absence of re-ranking), the specific change you made, and how you confirmed the fix actually worked — ideally with a before/after metric, not just a subjective 'it felt better.'

PROJECT & PORTFOLIO

Tell me about an agentic or tool-using system you built. What tools did it have access to, and how did you constrain its behavior?

Cover the tool set and why each tool was scoped the way it was, the reasoning/orchestration pattern used (ReAct, plan-and-execute, a fixed graph like LangGraph), and — this is the part interviewers listen for most closely — the specific guardrails you put in place: permission boundaries, human-confirmation steps for risky actions, loop/iteration limits, and how you handled tool failures. A project description that never mentions safety/failure handling reads as a demo, not production experience.

PROJECT & PORTFOLIO

Describe your evaluation approach for a GenAI feature you shipped. How did you know it was ready, and how did you monitor it after launch?

Walk through both offline and online evaluation: what your pre-launch eval set looked like and how you built it (real examples vs. synthetic, how you handled edge cases), which metrics you tracked and why they were the right ones for this specific feature, and what you monitored post-launch (user feedback signals, automated quality sampling, cost/latency). Bonus points for mentioning a case where evaluation caught a real regression before or after launch — it signals the evaluation process was actually load-bearing, not a formality.

PROJECT & PORTFOLIO

Tell me about the most complex prompt or context-engineering problem you had to solve.

Pick a problem with real constraints — conflicting requirements, a tight context budget, a task the model initially handled unreliably — and walk through your iteration process: what you tried first, why it fell short, what you changed, and how you validated the improvement (ideally against a concrete eval set rather than a handful of manual checks). This is a chance to show you iterate empirically on prompts like an engineer, not just intuitively.

PROJECT & PORTFOLIO

Describe a fine-tuning project you've done. What data did you use, what technique, and what were the results?

Cover: why fine-tuning was the right call for this problem (not RAG or prompting alone), how the dataset was sourced/cleaned and how large it was, the technique used (full fine-tune vs. LoRA/QLoRA) and why, and quantified before/after results on a held-out evaluation set. If you hit catastrophic forgetting, overfitting, or a data-quality issue along the way, naming it and how you addressed it is a stronger answer than a project that (implausibly) went smoothly end-to-end.

16

Behavioral & Judgment Questions

GenAI engineering roles are judged as much on judgment and communication as on technical depth. These questions probe how you think about failure, tradeoffs, and cross-functional work — prepare a real, specific story for each, not a generic answer.

BEHAVIORAL

Tell me about a time an AI system you built produced a wrong or harmful output. What did you do?

Use a structured (STAR) answer. Interviewers want to see: how you detected the failure (monitoring/eval, not luck), your root-cause process (did you separate retrieval vs. generation failure, was it a prompt issue vs. a data issue), the concrete fix you shipped, and what you changed structurally (a new eval case, a guardrail, a monitoring alert) so the same class of failure doesn't silently recur.

BEHAVIORAL

How do you decide when an LLM-based solution is the wrong tool for a problem?

Strong answers mention: tasks needing deterministic, auditable, 100%-reliable outputs (financial calculations, exact rule-based decisions) are often better served by traditional code/rules; tasks where a simpler heuristic or classical ML model achieves comparable accuracy at a fraction of the cost/latency; and tasks where hallucination risk is unacceptable and can't be sufficiently mitigated with grounding. Show judgment, not LLM-maximalism.

BEHAVIORAL

Walk me through how you'd estimate the cost of running a GenAI feature at scale before building it.

Estimate expected requests/day, average input+output tokens per request (including any RAG context or few-shot examples), multiply by the target model's per-token pricing, add embedding/vector-DB costs if RAG is involved, and stress-test the estimate against worst-case scenarios (long conversations, large retrieved context). Mention that this estimate should directly inform model-size/routing decisions, not be an afterthought post-launch.

BEHAVIORAL

How do you stay current in such a fast-moving field?

Be specific and genuine rather than generic — name concrete practices: following key research labs/papers, running small hands-on experiments with new techniques rather than only reading about them, participating in relevant communities, and critically evaluating new tools/frameworks against your own production experience rather than adopting hype uncritically.

BEHAVIORAL

Describe a project where you had to balance model quality against latency or cost constraints.

Walk through the actual tradeoff you faced, the options you considered (smaller model, caching, prompt optimization, quantization), how you measured the impact of each on both dimensions, and the decision framework you used to land on a final choice — ideally with a concrete number (e.g., '40% latency reduction for a 3% quality drop on our eval set').

BEHAVIORAL

How do you approach working with product/design teams who don't have technical AI backgrounds?

Emphasize translating technical constraints (hallucination risk, latency, cost-per-request) into product tradeoffs they can reason about, setting realistic expectations early about what LLMs can and can't reliably do, and building shared evaluation criteria together so 'is this good enough to ship' isn't a purely subjective judgment call made in isolation.

17

General Behavioral & HR Questions

Standard cross-role behavioral and HR-round questions that show up in most GenAI engineer loops alongside the technical rounds. These aren't GenAI-specific, but skipping prep for them is one of the most common reasons a technically strong candidate stumbles late in the process.

HR & GENERAL BEHAVIORAL

Tell me about a conflict you had with a coworker or manager and how you resolved it.

Pick a real, specific disagreement (technical direction, priorities, or approach) rather than a vague or interpersonal-drama story. Structure it as: what the disagreement actually was, how you approached the other person (did you seek to understand their reasoning before pushing your own), what the resolution was, and what you'd do differently now. Interviewers are listening for maturity and directness, not conflict avoidance or a story where you were simply right and the other person was wrong.

HR & GENERAL BEHAVIORAL

What's your biggest weakness, and what are you actively doing about it?

Avoid the classic non-answer disguised as a strength ('I work too hard'). Name a real, specific, professionally relevant weakness, then focus most of the answer on the concrete, ongoing action you're taking to address it — a specific habit, process, or feedback loop you've put in place — rather than a vague intention to 'get better.' A credible, still-in-progress growth story reads far better than a weakness that's suspiciously already fully solved.

HR & GENERAL BEHAVIORAL

Why do you want to work here, and why this role specifically?

Generic answers ('great culture,' 'exciting mission') are instantly forgettable. A strong answer connects something specific about the company's actual product, technical challenges, or stage to something specific about your background or what you want to grow into next — showing you've done real homework, not just applied broadly. If you can name a specific technical problem this team is likely facing and why it interests you, that's far stronger than generic enthusiasm.

HR & GENERAL BEHAVIORAL

Tell me about a time you failed. What did you learn from it?

Choose a genuine failure with real stakes and a clear consequence — not a humble-brag disguised as a failure. Walk through what went wrong, your role in it (own your part specifically, don't just describe circumstances), and most importantly, a concrete change in how you work now as a direct result. The learning should be specific and demonstrable, not a generic platitude like 'I learned to communicate better.'

HR & GENERAL BEHAVIORAL

How do you prioritize when everything feels urgent?

Strong answers describe an actual framework or heuristic you use — e.g., distinguishing genuinely urgent-and-important work from merely loud requests, assessing blast radius/impact if something slips, and communicating tradeoffs explicitly to stakeholders rather than silently trying to do everything. A concrete example where you had to explicitly deprioritize something (and how you communicated that) is much stronger than an abstract description of a prioritization philosophy.

HR & GENERAL BEHAVIORAL

Where do you see your career going over the next few years?

The interviewer is checking for realistic self-awareness and alignment with what this role can actually offer — not a rehearsed, overly specific title-and-timeline plan. A strong answer names the kind of technical depth or scope you want to grow into, connects it credibly to what this role/company can realistically provide, and avoids either extreme: no vision at all, or a trajectory that obviously outgrows this role within a year.

Quick-Reference Glossary

A last-minute skim sheet — the terms and formulas most likely to come up as rapid-fire follow-ups.

Attention formula	softmax(QK^T / sqrt(d_k)) V
Decoder-only models	GPT, LLaMA, Claude, Mistral — causal attention, autoregressive generation
Encoder-only models	BERT — bidirectional attention, best for classification/embeddings
RAG vs Fine-tuning	RAG = inject knowledge at query time. Fine-tune = change behavior/format/skill in weights
LoRA	Freeze base weights, train small low-rank adapter matrices (ΔW = BA)
QLoRA	LoRA + 4-bit quantized frozen base model — fine-tune large models on one GPU
Hybrid search	Dense (semantic) + Sparse/BM25 (exact match), fused via Reciprocal Rank Fusion
Re-ranking	Retrieve wide with bi-encoder → re-rank narrow with cross-encoder
RAGAS metrics	Faithfulness, Answer Relevance, Context Precision, Context Recall
KV cache	Cache Key/Value tensors of past tokens to avoid recomputation during generation
GQA	Grouped-Query Attention — shares K/V heads across query heads to shrink KV cache
Top-p vs Top-k	Top-p = dynamic cumulative-probability cutoff. Top-k = fixed candidate-pool size
ReAct	Reason → Act (tool call) → Observe → repeat, grounding reasoning in real feedback
Chinchilla law	Scale model size and training tokens roughly equally for compute-optimal training
LLM-as-judge risks	Position bias, self-preference bias, verbosity bias — validate against human labels

Before You Walk In

TECHNICAL PREP

- Be ready to draw the Transformer/attention diagram from memory and narrate it out loud.
- Practice the RAG-vs-fine-tuning-vs-prompting decision framework until it's automatic — it comes up in nearly every interview.
- Have one real project you can describe end-to-end: the problem, your architecture choices, what broke, how you evaluated it, and what you'd change.
- Know the actual cost/latency numbers for at least one model family you've used — vague answers here read as inexperience.

IN THE INTERVIEW

- When asked "how would you build X," always state assumptions and tradeoffs out loud — interviewers weight reasoning over a single correct answer.
- If you don't know something, say so plainly, then reason toward an answer from first principles — this is usually scored higher than confident guessing.
- For system design questions, structure your answer: requirements → architecture → tradeoffs → evaluation → failure modes.
- Ask clarifying questions before diving into a design — scale, latency needs, and data sensitivity change the right answer completely.
